1. Valid BST Permutations

A binary tree is a data structure characterized by the following properties:

- It can be an empty tree where root = null.
- It can consist of a root node that contains a value and two sub-trees labeled "left" and "right".
- Each sub-tree also follows the definition of a binary tree.

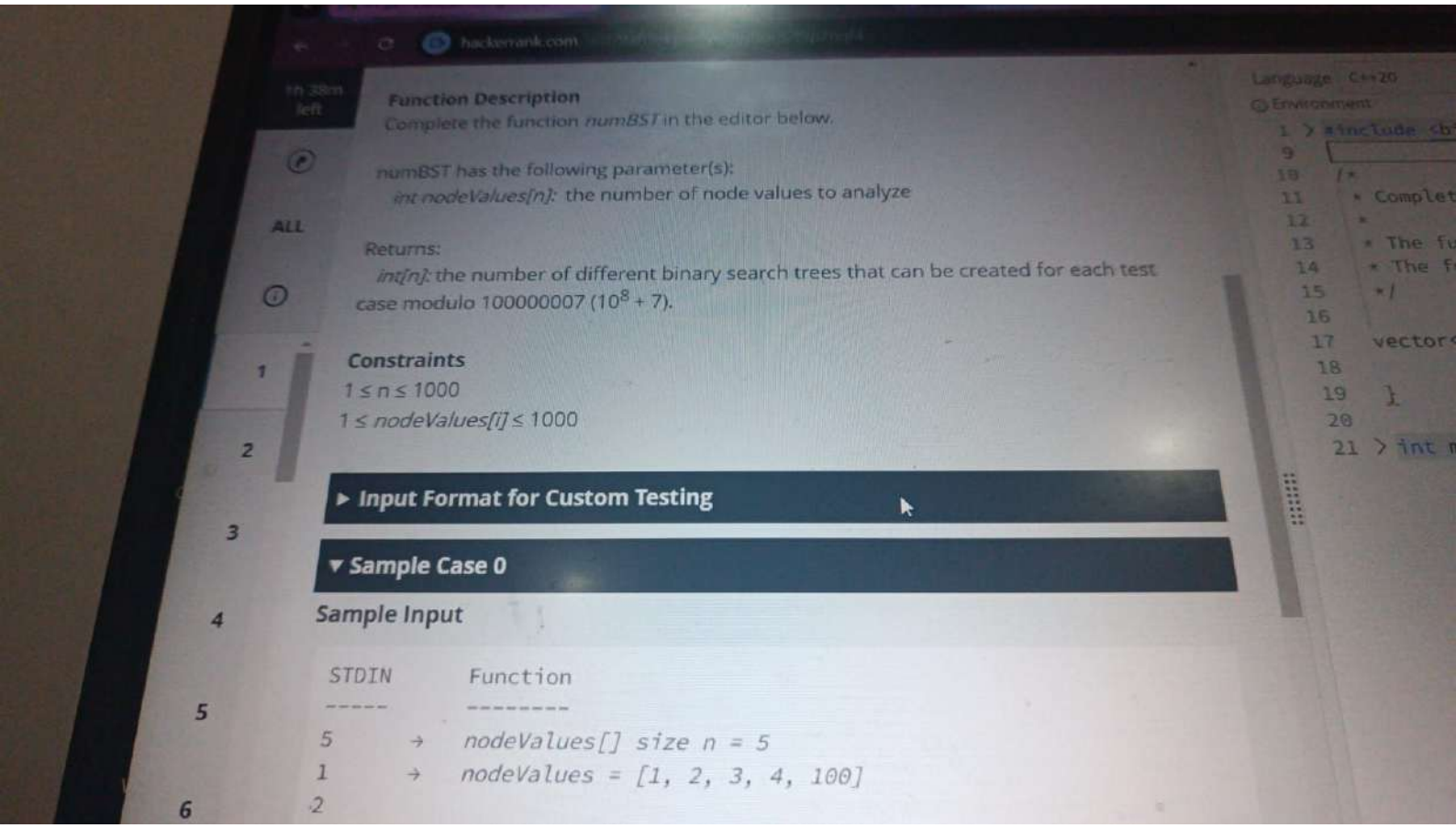
A binary tree is a binary search tree (BST) if all the non-empty nodes follow these two rules:

- If a node has a left sub-tree, then all the values in its left sub-tree are smaller than its value.
- If a node has a right sub-tree, then all the values in its right sub-tree are greater than its value.

Given an integer, determine the number of valid BSTs that can be created by nodes numbered from 1 to that integer. Please see the samples below for diagrams based on 1, 2 or 3 nodes.

Function Description

Complete the function *numBST* in the editor below.


hackerrank.com
1h 38m left
Function Description
Complete the function numBST in the editor below.
numBST has the following parameter(s):
int nodeValues[n]: the number of node values to analyze
ALL
Returns:
int[n]: the number of different binary search trees that can be created for each test case modulo 100000007 ($10^8 + 7$).
Constraints
$1 \le n \le 1000$
$1 \le nodeValues[i] \le 1000$
▸ Input Format for Custom Testing
▾ Sample Case 0
Sample Input
STDIN Function
5 → nodeValues[] size n = 5
1 → nodeValues = [1, 2, 3, 4, 100]
2
Language C++20
Environment

ALL

100

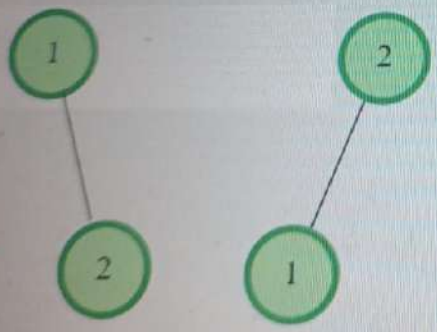
Sample Output

```
1
2
5
14
25666077
```

Explanation

Test Case #1: Only one tree can be created with one node.

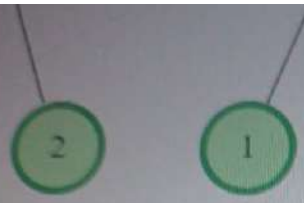
Test Case #2: Two trees can be created with two nodes.




```
12
13    * The f
14    * The f
15    */
16
17   vector<
18
19   }
20
21 > int m
```

2

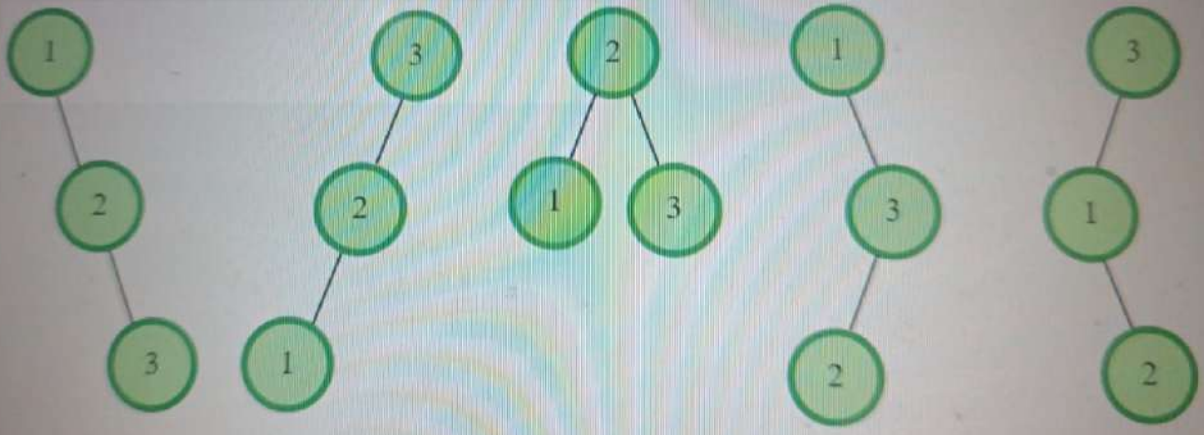
3



4

Test Case #3: Five trees can be created with three nodes.

5



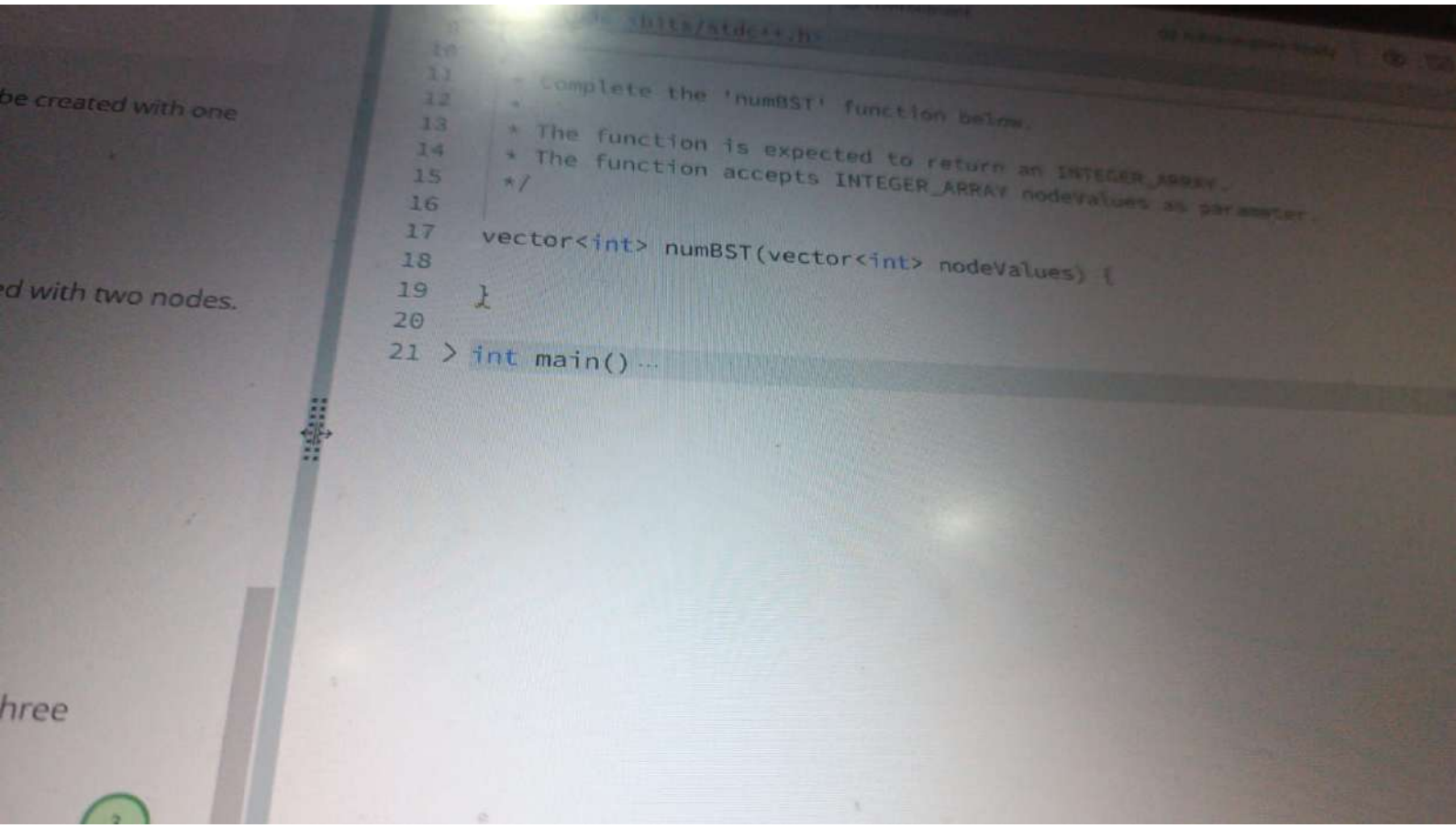
6

7

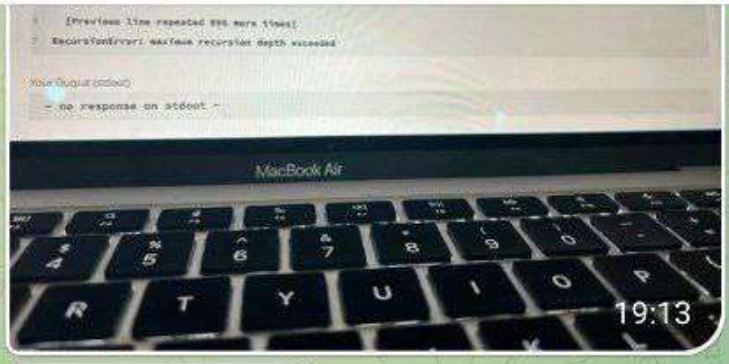
8

Sailing
Final result

Search


be created with one
ed with two nodes.
hree
bits/stdc++.h
Complete the 'numBST' function below.
* The function is expected to return an INTEGER_ARRAY.
* The function accepts INTEGER_ARRAY nodevalues as parameter.
vector<int> numBST(vector<int> nodeValues) {
}
int main()

```python
def bestSumAnyTreePath(parent, values):
    from collections import defaultdict

    n = len(parent)
    tree = defaultdict(list)
    for child in range(n):
        if parent[child] != -1:
            tree[parent[child]].append(child)

    max_sum = float('-inf')

    def dfs(node):
        nonlocal max_sum
        current_value = values[node]
        max_single = current_value
        child_max_single = []

        for child in tree[node]:
            child_max = dfs(child)
            child_max_single.append(child_max)
            max_single = max(max_single,
                current_value + child_max)

        if len(child_max_single) > 0:
            child_max_single.sort(reverse=True)
            max_path_here = current_value +
                child_max_single[0]
            if len(child_max_single) > 1:
                max_path_here = max
                    (max_path_here,
                    current_value +
                    child_max_single[0] +
                    child_max_single[1])
            max_sum = max(max_sum,
                max_path_here)

        max_sum = max(max_sum, current_value)
        return max_single

    dfs(0)
    return max_sum
```



Fine now 19:14

Ya phir se bheju pic 19:14

GT Broo only 30 mins left 19:16

```
def bestSumAnyTreePath(parent, values):
    from collections import defaultdict

    sys.setrecursionlimit(150000)
    n = len(parent)
    tree = defaultdict(list)
    for child in range(n):
        if parent[child] != -1:
            tree[parent[child]].append(child)

    max_sum = float('-inf')

    def dfs(node):
        nonlocal max_sum
        current_value = values[node]
        max_single = current_value
        child_max_single = []

        for child in tree[node]:
            child_max = dfs(child)
            child_max_single.append(child_max)
            max_single = max(max_single
```

1. Binary Manipulation

Given an integer, store its binary representation *bin* as a string or an array with digits indexed from *0* to *length(bin) - 1*. Reduce its binary representation to zero by using the following operations:

1. Change the i^{th} binary digit only if $(i+1)^{th}$ binary digit is 1 and all other binary digits from *(i+2)* to the end are zeros.
2. Change the rightmost digit without restriction.

What is the minimum number of operations required to complete this task?

```
1  > #include <assert.h> ···
19
20   /*
21    * Complete the 'minOperation
22    *
23    * The function is expected t
24    * The function accepts LONG_
25    */
26
27   long minOperations(long n) {
28
29   }
30
31 > int main() ···
```

Example

$n = 4$

The binary representation of 4 is 100 so for the example, *bin = [1, 0, 0]*.

```
100 → 101 → 111 → 110 → 010 → 011
→ 001 → 000
```

bin, from left to right, has indices 0 to 3. Stepping through the operations:

- Use rule 2 to change *bin[2]* to 1. 100 → 101
- Use rule 1 to change *bin[1]* to 1. 101 → 111
- Use rule 2 to change *bin[2]* to 0. 111 → 110
- Use rule 1 to change *bin[0]* to *0*. 110 → 010
- Use rule 2 to change *bin[2]* to *1*. 010 → 011
- Use rule 1 to change *bin[1]* to *0*. 011 → 001
- Use rule 2 to change *bin[2]* to *0*. 001 → 000

For this number, 7 operations are required to convert it to 0.

Note: In the binary representation of a number, the binary digit's positions are numbered as 0 to x-1 from left to right.

For this number, 7 operations are required to convert it to 0.

Note: In the binary representation of a number, the binary digit's positions are numbered as 0 to x-1 from left to right, where x is the number of digits in the binary representation of the number.

Function Description

Complete the function *minOperations* in the editor below.

minOperations has the following parameter:

long *n*: the starting value in base 10

Returns:

int: the minimum number of operations required to covert *n* to *0*

Constraints

- $1 \leq n \leq 10^{15}$

▶ Input Format For Custom Testing

▼ Sample Case 0

- $1 \leq n \leq 10^{15}$

▶ Input Format For Custom Testing

▼ Sample Case 0

Sample Input For Custom Testing

STDIN		Function
13	→	n = 13

Sample Output

```
9
```

Explanation

The binary representation of 13 is 1101. This is the sequence of the 9 operations that change 13 to 0 according to the constraints:

```
1101→1100→0100→0101→0111→0110→00
10→0011→0001→0000
```

▶ Sample Case 1

nguage C++20 Environment

```
1 > #include <bits/stdc++.h> ...
9
0   /*
1    * Complete the 'minOperations' functior
2    *
3    * The function is expected to return a
4    * The function accepts LONG_INTEGER n a
5    */
6
7   long minOperations(long n) {
8
9   }
0
1 > int main() ...
```

Test Results Custom Input

```cpp
int numTrees(int N)
{
    const int mod=1e9+7;
    vector<long long>dp(N+1,0);
    dp[0]=1;
    dp[1]=1;

    for(int i=2;i<=N;i++){
        for(int j=1;j<=i;j++){
            dp[i]=(dp[i]+((dp[j-1]%mod)*(dp[i-j]%mod))%mod)%mod;
        }
    }
    return dp[N]%mod;
}

vector<int> numBST(vector<int> nodeValues){
    vector<int> ans;
    for(int i=0 ; i<nodeValues.size() ; i++){
        ans.push_back(numTrees(nodeValues[i]));
    }
    return ans;
}
```